

馬鈴薯

- 時間限制：10 分鐘
- 環境：一張 GPU (≈16 GB VRAM)，無網際網路
- 程式大小：solution.ipynb ≤ 1 MB
- 儲存空間：5 GB

任務

你的朋友提議玩一個猜謎遊戲。他作為裁判，會從固定詞彙表中選擇一個隱藏詞，而你必須在最多 30 回合內找出它。在每回合中，裁判會對兩個詞進行比較，並回報哪一個在語意上更接近隱藏詞。每場遊戲都會由固定詞對 lamp vs potato 開始，因為它們是你朋友最喜歡的兩樣東西。接著，你的程式需要提出一個新詞。在每次比較當中的勝者會被保留，並與你的下一個提出的新詞進行比較。當你提出的詞與隱藏詞完全相同時，你便立即贏得該場遊戲。比較時不區分大小寫。你提出的每個詞都必須位於 dataset/vocabulary.json 中。

solution.ipynb 中提供了包含通訊協定與數據載入的完整範例。你可以對 PublicEmbeddingPlayer 類型進行修改。你的程式只會初始化一次，並在單次執行中進行所有遊戲；通訊協定會在每場遊戲開始時建立一個新的 PublicEmbeddingPlayer。

裁判

你的程式會向裁判傳送一個 JSON 對象，而裁判會以一個 JSON 對象回應。

以下是一個完整範例，其中僅為說明通訊協定而顯示隱藏詞：

```
Hidden word: shovel          Fixed opening: lamp vs potato

<- {"turn": 1, "winner_word": "potato", "verdict": "second", "word1": "lamp", "word2": "potato"}
-> {"new_word": "rock"}
<- {"turn": 2, "winner_word": "rock", "verdict": "second", "word1": "potato", "word2": "rock"}
-> {"new_word": "hammer"}
<- {"turn": 3, "winner_word": "hammer", "verdict": "second", "word1": "rock", "word2": "hammer"}
-> {"new_word": "shovel"}                                     <- matches: game won
-> {"status": "win"}
```

回合編號從 1 到 30。

verdict 的選項有：first，表示 word1 較接近隱藏詞；second，表示 word2 較接近；或者 same，表示兩個詞與隱藏詞的接近程度相同。

winner_word 是保留至下一次比較的詞。在裁定為 same 時，第一個詞會被保留。

Dataset

所有數據共用：

- dataset/vocabulary.json — 1602 個不重複的小寫詞。隱藏詞必定是其中之一。
- dataset/public_embeddings.npy — float32，形狀為 (1602, 2560)。第 i 行對應詞彙表中的詞 i。這些是公開的 embedding；裁判會使用不同的私有表示。

各資料分割都是隱藏詞的集合：

資料分割	詞數	答案	用途
test_public	120	✅ dataset/test_public.json	執行你的程式並自行評分
test_leaderboard_a	120	隱藏	即時排行榜
test_leaderboard_b	120	隱藏	最終排名

沒有 train 資料分割——不會使用帶標籤的資料列擬合任何內容。

提供的模型

本題隨附兩個可使用的預訓練 embedding 模型：

```
models/Qwen/Qwen3-Embedding-0.6B
[Qwen Embedding model card] (https://yastatic.net/s3/contest/ioai/3/01_Qwen_Qwen3-Embedding-0.6B_MODEL_CARD.html)
models/BAAI/bge-m3
[bge-m3 model card] (https://yastatic.net/s3/contest/ioai/3/02_BAAI_bge-m3_MODEL_CARD.html)
```

兩者都必須從其本機路徑載入；使用 Hugging Face Hub ID，例如 "BAAI/bge-m3"，會觸發下載並失敗，因為評測環境離線。每個目錄都包含一個可執行的 `example.py`，示範離線呼叫方式。

可用的函數庫：`numpy`、`torch`、`sentence-transformers`。無網際網路、不可下載，也沒有其他套件。

輸出

無。這是一道互動式題目：你的程式不會寫入答案檔案；它會依照上述方式透過 `stdin/stdout` 與裁判通訊。

評分指標

在第 t 回合找到答案的遊戲得分為 $1.0 - 0.02 \times \max(0, t - 10)$ ；未能在 30 回合內解出的遊戲得分為 0。因此，第 1-10 回合的得分為 1.00，第 20 回合的得分為 0.80，第 30 回合的得分為 0.60。

你的任務分數為遊戲平均得分 $\times 100$ ，介於 0.00 與 100.00 之間。

10 分鐘的限制是涵蓋啟動、準備，以及測試集中所有 120 場遊戲的單一總時間預算。

如何提交

1. 開啟 `solution.ipynb`、編輯 `PublicEmbeddingPlayer`，並執行所有儲存格以確認其正常運作。
2. 你也可以選擇在本機檢查：`python local_test.py solution.ipynb --limit 5`。本機裁判使用公開的 embedding，因此其分數 僅供參考。
3. 儲存 `solution.ipynb`。
4. 開啟 JupyterLab 左側邊欄中的 Git 分頁。
5. 暫存 `solution.ipynb`（其旁邊的 + 圖示）。
6. 輸入一則 commit 訊息，然後按一下 Commit。
7. 按一下帶有向上箭頭的雲朵圖示以推送。
8. 返回此 Contest 頁面並按一下 Submit，且 commit 訊息須與你所提供的訊息相符。

請只提交一個名為 `solution.ipynb` 的檔案，其中應涵蓋任何必要的準備與推論。